

# Day 2 Backend Plan

## Contents

---

01  CivicFlow-AI — Day 2 Complete Interview Notes Today ... RESPONSE

## CivicFlow-AI — Day 2 Complete Interview Notes

Today you moved from the **CRUD foundation** toward a more **production-ready FastAPI backend**.

The major topics covered were:

text

```
Environment Variables
↓
.env + python-dotenv
↓
Git Security
↓
Alembic
↓
Database Migrations
↓
Exception Handling
↓
Custom Exceptions
↓
Global Exception Handler
↓
Async FastAPI
↓
Interview Preparation
```

---

# 1. Environment Variables

## What is an environment variable?

An environment variable is a configuration value stored outside the application source code.

For example:

```
text
```

```
DATABASE_URL=postgresql+psycopg2://postgres:YOUR_PASSWORD@localhost:5432/civicflow
```

Instead of writing the database credentials directly inside Python:

```
python
```

```
DATABASE_URL = "postgresql+psycopg2://postgres:password@localhost:5432/civicflow"
```

we use:

```
python
```

```
import os
from dotenv import load_dotenv

load_dotenv()

DATABASE_URL = os.getenv("DATABASE_URL")
```

## Why use environment variables?

Main reasons:

1. Security
2. Configuration management
3. Different environments can use different values
4. Secrets don't need to be hardcoded

For example:

```
text
```

```
Development → Local PostgreSQL  
Testing      → Test database  
Production   → Production database
```

The code can remain the same while the configuration changes.

---

## 2. `.env` File

A `.env` file is commonly used to store environment-specific configuration.

Example:

```
text
```

```
DATABASE_URL=postgresql+psycopg2://postgres:YOUR_PASSWORD@localhost:5432/civicfl
```

Python reads it using:

```
python
```

```
load_dotenv()
```

and retrieves values using:

```
python  
  
os.getenv("DATABASE_URL")
```

## Complete flow

```
text  
  
.env  
↓  
load_dotenv()  
↓  
Environment  
↓  
os.getenv()  
↓  
DATABASE_URL  
↓  
SQLAlchemy Engine  
↓  
PostgreSQL
```

---

## 3. Git Security

Sensitive configuration should not be committed to Git.

Your `.gitignore` should contain:

```
text
```

```
.env  
__pycache__/  
*.pyc
```

## Why `.env`?

Because it may contain:

```
text
```

```
Database password  
API keys  
Secret keys  
Tokens  
Other credentials
```

## Why `__pycache__`?

Python automatically generates cache files such as:

```
text
```

```
__pycache__/  
*.pyc
```

These are generated files and normally don't belong in the source repository.

## Useful command

To verify that `.env` is ignored:

```
powershell
```

```
git check-ignore -v .env
```

## 4. Alembic

### What is Alembic?

*"Alembic is a database migration tool used with SQLAlchemy to manage and track database schema changes."*

Suppose your current table is:

```
text
reports
├ id
├ title
├ description
└ location
```

Then you modify your SQLAlchemy model:

```
text
reports
├ id
├ title
├ description
├ location
└ severity
```

Alembic allows you to represent that change as a migration.

---

## 5. Why Database Migrations?

Without migrations, developers might manually modify the database every time the model changes.

With Alembic:

```
text

SQLAlchemy Model
    ↓
Schema Change
    ↓
Alembic Migration
    ↓
Database
```

This provides a history of database schema changes.

---

## 6. Alembic Setup

You initialized Alembic using:

```
powershell

alembic init alembic
```

This created:

```
text
```

```
backend/
├── alembic/
│   ├── versions/
│   ├── env.py
│   ├── README
│   └── script.py.mako
└── alembic.ini
```

The important directory is:

```
text

alembic/versions/
```

because migration files are stored there.

---

## 7. `alembic revision --autogenerate`

Command:

```
powershell

alembic revision --autogenerate -m "Migration message"
```

It compares the SQLAlchemy model metadata with the database schema and generates a migration containing detected changes.

### Important interview point

Autogenerate is helpful, but you should **review the generated migration before applying it**.

It detects schema differences, but it doesn't necessarily know the correct business/data migration strategy.

---

## 8. `alembic upgrade head`

Command:

```
powershell  
  
alembic upgrade head
```

means:

*“Apply all pending migrations until the latest migration revision.”*

Your migration chain became:

```
text  
  
4ea58ba5048f  
  ↓  
64e1b91d61dc  
  ↓  
HEAD
```

---

## 9. `alembic current`

Command:

```
powershell  
  
alembic current
```

shows the migration revision currently applied to the database.

Your final output:

```
text  
  
64e1b91d61dc (head)
```

means your database is currently at the latest migration.

---

## 10. Important Migration Problem You Solved 🔥

You added:

```
python  
  
severity: Mapped[str] = mapped_column(  
    String(20),  
    nullable=False,  
    default="MEDIUM"  
)
```

Alembic initially generated:

```
python
```

```
op.add_column(  
    'reports',  
    sa.Column(  
        'severity',  
        sa.String(length=20),  
        nullable=False  
    )  
)
```

But existing reports were already present in the database.

Therefore PostgreSQL couldn't add a new `NOT NULL` column because existing rows would initially have:

```
text
```

```
severity = NULL
```

This caused:

```
text
```

```
psycopg2.errors.NotNullViolation
```

---

## 11. Safe Migration Strategy

You corrected the migration to:

### Step 1 — Add nullable column

python

```
op.add_column(  
    'reports',  
    sa.Column(  
        'severity',  
        sa.String(length=20),  
        nullable=True  
    )  
)
```

## Step 2 — Backfill existing rows

python

```
op.execute(  
    "UPDATE reports SET severity = 'MEDIUM' WHERE severity IS NULL"  
)
```

## Step 3 — Make it NOT NULL

python

```
op.alter_column(  
    'reports',  
    'severity',  
    existing_type=sa.String(length=20),  
    nullable=False  
)
```

## Final flow

text

Existing Database

↓

Add severity as nullable

```
↓  
Existing rows → MEDIUM  
↓  
Change severity → NOT NULL  
↓  
Migration successful
```

## Interview answer

*"When adding a new non-nullable column to a table containing existing data, we need to safely handle the existing rows. We can initially add the column as nullable, populate existing rows with an appropriate value, and then change the column to NOT NULL."*

# 12. Python Default vs Database Migration

This is an important concept.

Your model has:

```
python  
  
default="MEDIUM"
```

That does **not automatically** mean that existing database rows will be populated when a **migration adds the column**.

That's why your migration needed:

```
python  
  
op.execute(  
    "UPDATE reports SET severity = 'MEDIUM' WHERE severity IS NULL"
```

)

## Interview point

*“Application-level defaults and database migration/backfill logic are separate concerns. Existing rows must be explicitly considered during schema changes.”*

# 13. HTTPException

FastAPI provides:

```
python

from fastapi import HTTPException
```

Example:

```
python

raise HTTPException(
    status_code=404,
    detail="Report not found"
)
```

It allows the API to return an HTTP error response.

# 14. `raise` vs `return`

## Correct

python

```
raise HTTPException(  
    status_code=404,  
    detail="Report not found"  
)
```

## Incorrect

python

```
return HTTPException(  
    status_code=404,  
    detail="Report not found"  
)
```

Why?

Because `HTTPException` is an exception.

`raise`:

text

```
Stops normal execution  
↓  
FastAPI handles exception  
↓  
HTTP error response
```

`return` means:

text

Normal function result

## Interview answer

*"We use `raise` because `HTTPException` is an exception. `raise` stops normal execution and allows FastAPI to generate the appropriate HTTP error response."*

# 15. HTTP Status Codes

Important codes from today's session:

| Status | Meaning               |
|--------|-----------------------|
| 200    | OK                    |
| 201    | Created               |
| 400    | Bad Request           |
| 401    | Unauthorized          |
| 403    | Forbidden             |
| 404    | Not Found             |
| 500    | Internal Server Error |

## 404

Used when the requested resource doesn't exist.

Example:

text

```
GET /api/reports/999
```

If report `999` doesn't exist:

```
text
```

```
404 Not Found
```

## 500

Used when an unexpected server-side problem occurs.

Example:

```
text
```

```
Unexpected database/server failure
```

```
↓
```

```
500 Internal Server Error
```

## Interview answer

*"404 means the requested resource could not be found, while 500 indicates an unexpected server-side failure while processing the request."*

# 16. Your Existing Report Error

Your original service logic was:

python

```
def get_report_by_id(db: Session, report_id: int):  
    report = db.query(Report).filter(Report.id == report_id).first()  
  
    if report is None:  
        raise HTTPException(  
            status_code=404,  
            detail="Report not found"  
        )  
  
    return report
```

This is valid FastAPI code.

---

## 17. Custom Exceptions

You then learned a more scalable architecture.

You created:

text

```
app/  
└─ exceptions/  
    ├── __init__.py  
    └─ report_exceptions.py
```

Inside:

```
python
```

```
class ReportNotFoundException(Exception):  
    pass
```

This is an application-specific exception.

It means:

```
text
```

```
ReportNotFoundException  
    ↓  
Requested report doesn't exist
```

---

## 18. Service Layer with Custom Exception

Instead of:

```
python
```

```
raise HTTPException(  
    status_code=404,  
    detail="Report not found"  
)
```

your service now uses:

```
python
```

```
from app.exceptions.report_exceptions import ReportNotFoundException
```

and:

```
python

if report is None:
    raise ReportNotFoundException()
```

So:

```
text

Service
  ↓
ReportNotFoundException
```

The service doesn't need to decide the HTTP response.

---

## 19. Global Exception Handler

FastAPI allows us to define an exception handler:

```
python

@app.exception_handler(ReportNotFoundException)
async def report_not_found_handler(
    request: Request,
    exc: ReportNotFoundException
):
    return JSONResponse(
        status_code=404,
```

```
        content={
            "detail": "Report not found"
        }
    )
```

Now the architecture becomes:

```
text

Client
  ↓
Router
  ↓
Service
  ↓
ReportNotFoundException
  ↓
Global Exception Handler
  ↓
JSONResponse
  ↓
404 Not Found
```

---

## 20. Why Custom Exceptions?

### Direct approach

```
text

Service
  ↓
HttpException
  ↓
HTTP response
```

## Custom exception approach

```
text

Service
  ↓
Business/Application Exception
  ↓
Global Exception Handler
  ↓
HTTP Response
```

The second approach separates:

```
text

Business logic
```

from:

```
text

HTTP handling
```

## Interview answer

*“Custom exceptions help separate application or business errors from HTTP-specific response handling. The service raises an application-specific exception, and a global exception handler converts it into the appropriate HTTP response.”*

# 21. Async Python

You also learned:

```
python
```

```
def
```

versus:

```
python
```

```
async def
```

**`def`**

Defines a normal synchronous function.

```
python
```

```
def get_report():  
    return "Report"
```

**`async def`**

Defines a coroutine that can perform asynchronous operations.

```
python
```

```
async def get_report():  
    ...
```

## 22. `await`

`await` is used inside `async def`.

Example:

python

```
async def get_data():  
    result = await some_async_operation()  
    return result
```

Conceptually:

text

```
async def  
  ↓  
Coroutine  
  ↓  
await asynchronous operation  
  ↓  
Event loop can handle other available tasks
```

---

## 23. Why Async?

Async programming is particularly useful for **I/O-bound operations**.

Examples:

text

```
Network requests
External API calls
Asynchronous database operations
Other I/O operations
```

Imagine:

```
text

Request A
  ↓
Waiting for network
```

During that waiting period, asynchronous execution can allow another available task to make progress.

---

## 24. `time.sleep()` vs Async Waiting

Synchronous example:

```
python

import time

def task():
    time.sleep(3)
```

Async example:

```
python
```

```
import asyncio

async def task():
    await asyncio.sleep(3)
```

Conceptually:

```
text

time.sleep()
  ↓
blocks the current synchronous execution

await asyncio.sleep()
  ↓
asynchronous waiting
```

You practiced this using `asyncio.gather()`.

---

## 25. `asyncio.gather()`

You practiced multiple asynchronous tasks:

```
python

await asyncio.gather(
    task_one(),
    task_two()
)
```

This allows the async tasks to be scheduled together.

For example:

text

Task 1 → waits 3 seconds

Task 2 → waits 2 seconds

Both start

↓

Task 2 finishes

↓

Task 1 finishes

This demonstrated the basic idea of asynchronous concurrency.

---

## 26. FastAPI and Async

An async FastAPI endpoint can look like:

python

```
@app.get("/async-test")
async def async_test():
    await asyncio.sleep(2)

    return {
        "message": "Async operation completed"
    }
```

## 27. Important: Don't blindly use `async def`

This was one of today's most important concepts.

Your current CivicFlow-AI database code uses synchronous SQLAlchemy:

```
python
```

```
def get_all_reports(db: Session):  
    return db.query(Report).all()
```

Changing only:

```
python
```

```
def
```

to:

```
python
```

```
async def
```

doesn't automatically make:

```
python
```

```
db.query(Report).all()
```

asynchronous.

## Synchronous architecture

```
text
```

```
FastAPI
  ↓
def
  ↓
Sync SQLAlchemy Session
  ↓
PostgreSQL
```

## Fully asynchronous architecture

```
text

FastAPI
  ↓
async def
  ↓
Async SQLAlchemy Session
  ↓
Async PostgreSQL driver
```

## Interview answer

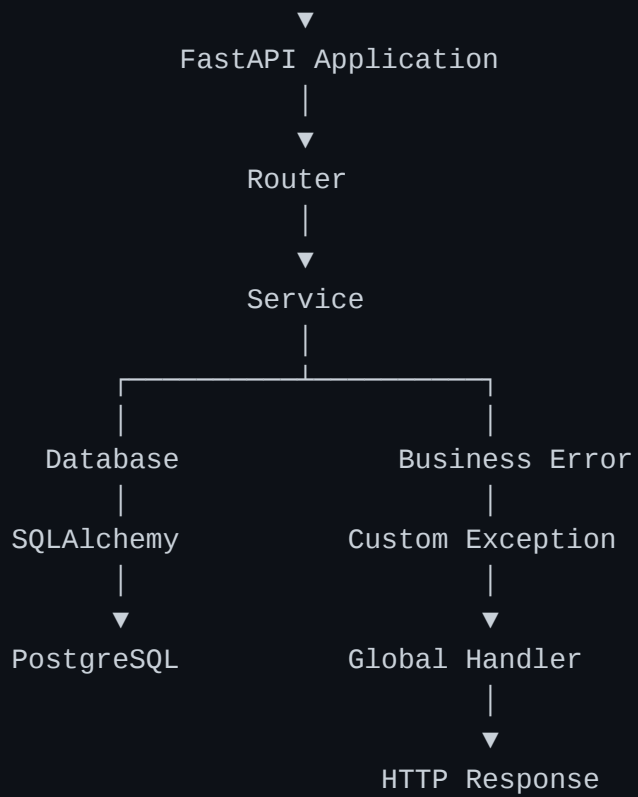
*"We should not convert every FastAPI function to `async def`. We should use async when the operations inside the function are asynchronous. Simply changing `def` to `async def` does not make synchronous database operations asynchronous."*

# 28. Complete Day 2 Architecture

You now understand a much more production-oriented backend:

```
text

CivicFlow-AI
|
```



And configuration:

```
text

.env
↓
load_dotenv()
↓
os.getenv()
↓
DATABASE_URL
↓
SQLAlchemy
↓
PostgreSQL
```

And database evolution:

```
text

SQLAlchemy Model
↓
```

```
Alembic Autogenerate
↓
Migration Script
↓
Review Migration
↓
alembic upgrade head
↓
PostgreSQL Schema
```

# Most Important Interview Questions

If you have limited time before an interview, memorize these first.

## 1. What is Alembic?

*"Alembic is a database migration tool used with SQLAlchemy to track and apply database schema changes."*

## 2. What is `HTTPException`?

*"It is used in FastAPI to stop normal request processing and return an HTTP error response with a specified status code and detail."*

## 3. Why `raise` instead of `return`?

*"Because HTTPException is an exception, and `raise` stops normal execution and lets FastAPI handle the error response."*

## 4. Difference between 404 and 500?

*"404 means the requested resource doesn't exist, while 500 means an unexpected server-side failure occurred."*

## 5. What is a custom exception?

*"It is an application-specific exception created to represent a particular business or application error."*

## 6. Why use a global exception handler?

*"It converts application-specific exceptions into appropriate HTTP responses while keeping business logic separate from HTTP handling."*

## 7. What is ``async def``?

*"It defines a coroutine that can perform asynchronous operations using ``await``."*

## 8. What does ``await`` do?

*"It waits for an asynchronous operation to complete while allowing the event loop to handle other available tasks."*

## 9. Should every FastAPI function be async?

*"No. Async should be used when the operations are asynchronous. Simply changing a synchronous function to ``async def`` doesn't make synchronous operations asynchronous."*

## 10. Why use ``.env``?

*"To keep configuration and sensitive values such as database credentials outside the source code."*



# 30-Second Day 2 Revision

Before an interview, remember this:

text

`.env`  
→ stores configuration/secrets

`load_dotenv()`  
→ loads `.env`

`os.getenv()`  
→ reads environment variables

`.gitignore`  
→ prevents sensitive/generated files from Git

Alembic  
→ database migrations

`revision --autogenerate`  
→ detects schema changes

`upgrade head`  
→ applies pending migrations

`current`  
→ shows current migration

`HTTPException`  
→ FastAPI HTTP error

`raise`  
→ raises exception and stops normal flow

`404`  
→ resource not found

`500`  
→ unexpected server error

Custom Exception  
→ application-specific error

Global Handler  
→ converts exception → HTTP response

`async def`  
→ asynchronous coroutine

`await`  
→ waits for async operation

Async  
→ useful for I/O-bound operations

Important:  
async def ≠ automatically asynchronous database code

## Day 2 Achievement

You progressed from:

text

CRUD API

to:

text

CRUD

↓

Environment Configuration

↓

Git Security

↓

Database Migrations

↓

Safe Schema Evolution

↓

Exception Handling

↓

Custom Exceptions

↓

Global Exception Handling

↓

Async Programming Concepts

↓

Interview-Ready Backend Knowledge

This is a solid **Day 2 FastAPI/backend interview foundation**.